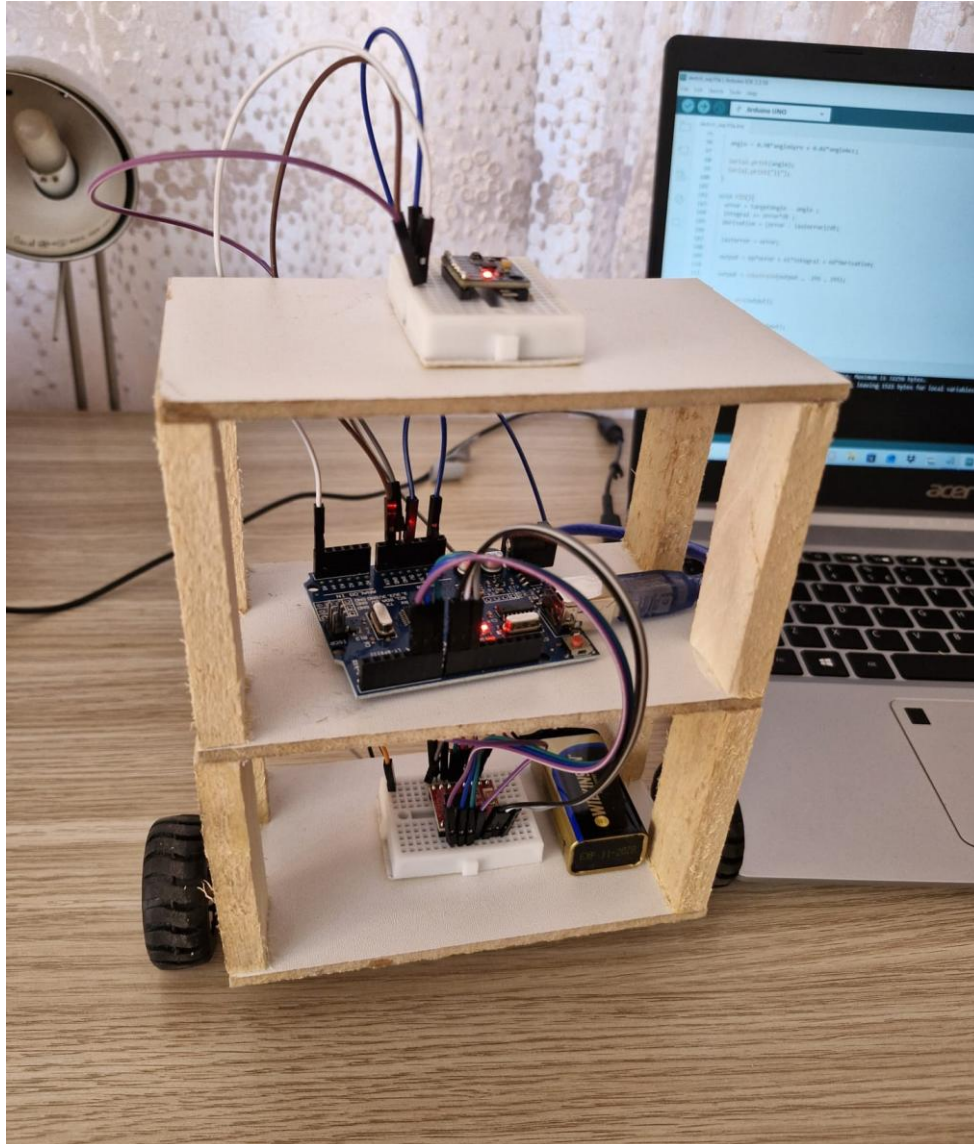




Self Balancing Robot

ARDUINO / MPU6050/ PID

EFE GULBA | YTU - Control and Automation Engineering | 12.08.2026

Project Objective

The objective of this project is to design and develop a self balancing robot capable of saving its vertical position on two wheels. The robot uses an MPU6050 (IMU) to measure its tilt angel and Arduino UNO to process the sensor data. A PID control algorithm is used to calculate the needed motor response and continuously correct the robot's position.

The main goal of the project is to understand and apply the fundamental concepts of sensor data processing , feedback control , PID control , PWM motor control and embedded systems in a real world application.

Materials

- Arduino Uno
- MPU6050
- TB6612FNG Motor Driver
- 2 DC Motors and 2 Wheels
- 2 Batteries (9V)
- Jumper cables and Breadboard

System

MPU6050 ---> Arduino ---> PID ---> Motor Driver ---> Motors ---> Robot

MPU6050 and Calculating the Angle

Mpu6050 sensor contains a 3 axis accelerometer and a 3 axis gyroscope which provide information about robot's movement and rotation.

First, the accelerometer data was used to calculate the tilt angle using atan() function. Then, the gyroscope was used to measure the angular velocity of the robot. By integrating gyroscope data over time , the change in the angle could be calculated

To obtain a more reliable angle measurement, a complementary filter was used . The filter combines the accelerometer and gyroscope measurements.

$$\text{Angle} = 0.98 * (\text{Gyroscope Angle}) + 0.02(\text{Accelerometer Angle})$$

During testing, the sensor produced approximately 44.5 degrees when the robot was tilted forward 45 degrees , and similar negative value when tilted backward.

PID Control

A PID controller was implemented to keep the robot balanced on two wheels. The controller calculates the error between the desired angle and measured angle from MPU.

The PID controller consists of three components;

P: Responds to the current error

I: Reduces steady state error

D: Reduces oscillations and improves stability

PID Output = $K_p \cdot \text{error} + K_i \cdot \text{Integral} + K_d \cdot \text{Derivative}$

The output is converted into a PWM signal to control the motors. Different K_p , K_i , K_d values were tested to achieve stable balancing.

Conclusion

The self balancing robot was successfully developed using an sensor , microcontroller , motor driver and PID control algorithm.

The sensor data processing , complementary filter and PID control were developed without using libraries except MPU6050.h and Wire.h

The robot can maintain its balance on two wheels and recover its position after external impacts. Although small vibrations remain , the main objective of achieving stable balancing and disturbance recovery was successfully achieved.

This project provided practical experience in sensor data processing , control algorithms , PWM motor control and embedded systems.

Future Improvements

- Better Batteries: Higher quality batteries with better power delivery can provide more stable motor performance and longer operating time.
- PID Tunning: Better PID parameters can reduce the remaning vibrations and improve the robot's stability.
- Mechanical Design: Mechanical structure can be improved by using better materials.

Project Videos

- <https://drive.google.com/file/d/1BMdbfLBVHo3XaPT3UsrbKrJg59-BPtNS/view?usp=drivesdk>
- https://drive.google.com/file/d/1ODbm0iiNkW_d5G4FuTd38rDd4IrDgtXS/view?usp=drivesdk